

LLM-MODUL 2

DIE NÄCHSTE RUNE



Hafen



Langhaus



Wasserfall



Bergpass



Hafen





BJÖRN CYBERVIKING

DER RUNENSTEIN UND UND NÄCHSTE RUNE

Eine Frage. Viele Möglichkeiten. Eine Rune nach der anderen.

SI-LLM-02 – Next Token Prediction



WISSEN
SCHÜTZT
UNSERE
ZUKUNFT

1

DIE FRAGE

Orakel,
welcher Weg zum Hafen
ist der sicherste?

Ich schaue
in die Runen ...



2

DIE MÖGLICHEN NÄCHSTEN RUNEN

Welcher Weg zum Hafen?



Hafen



Langhaus



Bergpass



Wasserfall

Aus dem bisherigen Kontext
entstehen mögliche nächste
Runen (Tokens).

3

DER NEUE KONTEXT – NEUE MÖGLICHKEITEN



Hafen

→ IST →

DER →

...



schnellste



sicherste



steilste



längste

Der bisherige Kontext beeinflusst
die nächsten möglichen Runen.

4

EINE RUNE WIRD AUSGEWÄHLT



→ IST →

DER →



sicherste

Die gewählte Rune wird
Teil des Kontextes.

5

DIE ANTWORT WÄCHST WEITER



→ IST →

DER →



→ ...



WEG



NACH



DURCH



ZUR

Mit jeder neuen Rune entstehen
wieder neue Möglichkeiten.

6

DIE ERKENNTNIS

Du denkst also
nicht die ganze Antwort
auf einmal?

Nein, Björn.
Eine Rune nach
der anderen.



→ IST →

DER →



Der Weg zum Hafen ist der sicherste ...

So entstehen aus
vielen kleinen Schritten
große Antworten.

MERKE:

Ein LLM wählt nicht die ganze Antwort auf einmal.
Es erzeugt Schritt für Schritt die nächste Rune (Token)
basierend auf dem bisherigen Kontext.

Eine gute Frage
führt zu vielen Wegen.
Die richtige Entscheidung
zu einer klaren Antwort.





TEXT
WIRD ZU
TOKENS



TOKENS & PREDICTION

DIE NÄCHSTE RUNE

Wie aus Kontext Schritt für Schritt
die nächste Rune (Token) entsteht.



EINE RUNE
NACH DER
ANDEREN

1 WAS IST EIN TOKEN?

Ein Token ist eine Texteinheit, die von einem LLM verarbeitet wird. Ein Token kann ein ganzes Wort, ein Teil eines Wortes, ein Satzzeichen oder ein anderer Textbaustein sein.



2 WORT $\neq$ TOKEN

Ein Wort kann aus mehreren Tokens bestehen. Die Aufteilung hängt vom verwendeten Tokenizer ab.
(Beispiel vereinfacht)



3 TOKENISIERUNG

Ein Tokenizer zerlegt den Text in Tokens. Er kennt Regeln und ein Vokabular, das häufige Textmuster effizient abbildet.



4 VOM TOKEN ZUR ZAHL

Jeder Token wird durch eine eindeutige numerische ID dargestellt. Diese Zahlen werden in Vektoren (Embeddings) umgewandelt, die das Modell verarbeiten kann. So kann das Modell die Bedeutung und den Zusammenhang der Tokens verstehen.



5 NEXT TOKEN PREDICTION

Das Modell bewertet auf Basis des bisherigen Kontexts alle möglichen nächsten Tokens und ordnet ihnen eine Wahrscheinlichkeit zu.



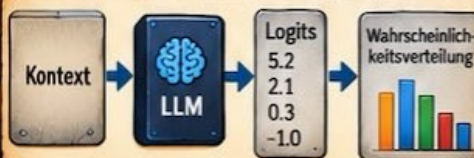
6 DER KONTEXT ENTSCHIEDET MIT

Die bisherigen Tokens (der Kontext) beeinflussen, welche nächsten Tokens wahrscheinlich sind.



7 LOGITS & WAHRSCHEINLICHKEIT

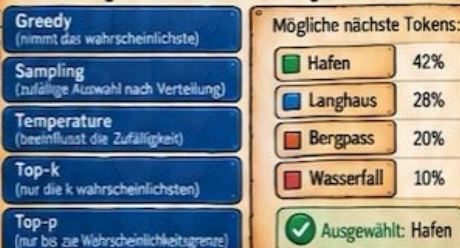
Das Modell erzeugt für alle möglichen Tokens Logits – eine Art Punktzahl. Diese werden in eine Wahrscheinlichkeitsverteilung umgewandelt.



Die Wahrscheinlichkeit ist keine Wahrheitprüfung, sondern eine statistische Schätzung.

8 DECODING

Beim Decoding wird aus den bewerteten Tokens ein nächstes Token ausgewählt. Es gibt verschiedene Strategien:



9 TOKEN FÜR TOKEN

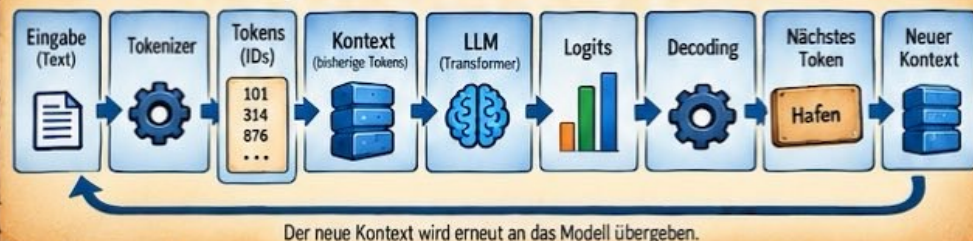
Die Antwort entsteht Schritt für Schritt. Das ausgewählte Token wird Teil des neuen Kontexts und beeinflusst die nächsten Tokens.



Jedes neue Token erweitert den Kontext.

10 DIE GESAMTE KETTE

Von der Eingabe bis zur nächsten Rune – ein wiederkehrenden Prozess.



Der neue Kontext wird erneut an das Modell übergeben.

11 PRAXIS FÜR DIE SYSTEMINTEGRATION

Wichtige Aspekte im praktischen Einsatz von LLMs:

- Context Window – begrenzte Länge des Kontexts
- Rechenleistung – GPU/CPU, Skalierung, Infrastruktur.
- Latenz – Antwortzeit je nach Modell und Umgebung
- API-Kosten / Tokenverbrauch – Abrechnung pro Token
- RAG-Kontext – mit eigenen Daten sinnvolle Antworten
- Inferenz-Infrastruktur – lokal oder in der Cloud

MERKE:

Ein LLM erzeugt Text nicht Wort für Wort, sondern Token für Token.
Tokenisierung zerlegt Text. Prediction bewertet mögliche nächste Tokens.
Decoding wählt eines davon aus. Das gewählte Token wird Teil des neuen Kontexts.

SI-LLM-02 – Next Token Prediction

Der Runenstein und die nächste Rune

1. Einordnung des Moduls

Im Modul SI-LLM-01 wurde vermittelt, dass Large Language Models Sprache verarbeiten und auf Grundlage des während des Trainings gelernten Wissens neue Texte erzeugen. Die vereinfachte Vorstellung „Wort für Wort“ ist für den Einstieg hilfreich, beschreibt die technische Verarbeitung jedoch nicht genau genug.

Dieses Modul betrachtet deshalb den nächsten Verarbeitungsschritt: Ein LLM arbeitet nicht direkt mit menschlichen Wörtern, sondern mit Tokens. Der Eingabetext wird zunächst tokenisiert, anschließend in eine numerische Darstellung überführt und vom Modell verarbeitet. Das Modell berechnet mögliche Fortsetzungen für den gegebenen Kontext. Durch einen Decoding-Schritt wird daraus das nächste Token ausgewählt. Dieses Token wird Teil des neuen Kontexts und der Vorgang wiederholt sich.

Zentrale Frage des Moduls: Wie wird aus einer Eingabe Schritt für Schritt eine Antwort, wenn ein LLM nicht mit Wörtern, sondern mit Tokens arbeitet?

2. Lernziele

Nach Abschluss des Moduls können die Lernenden:

- den Begriff Token erklären und von einem menschlichen Wort unterscheiden.
- erklären, dass ein Wort aus mehreren Tokens bestehen kann.
- die Aufgabe eines Tokenizers beschreiben.
- eine vereinfachte Tokenisierung nachvollziehen.
- erklären, dass Tokens für die Verarbeitung in eine numerische Darstellung überführt werden.
- das Prinzip der Next Token Prediction erklären.
- beschreiben, welchen Einfluss der vorhandene Kontext auf die Vorhersage hat.
- erklären, dass das Modell mehrere mögliche nächste Tokens mit unterschiedlichen Wahrscheinlichkeiten bewerten kann.
- die Begriffe Logits, Wahrscheinlichkeitsverteilung und Decoding auf grundlegender Ebene einordnen.
- erklären, warum bei gleichem Eingabetext unterschiedliche Ausgaben entstehen können.
- den Zusammenhang zwischen Tokenisierung, Prediction und Generation beschreiben.

3. Didaktische Ausgangslage

Die Analogie des Orakelwikingers aus Modul 1 wird weitergeführt. Der Runenstein steht weiterhin für das LLM. Die einzelnen Runen dienen als anschauliches Bild für Tokens.

Dabei ist wichtig: Eine Rune entspricht nicht zwingend einem ganzen Wort. Wie bei echten Tokenizern kann eine Einheit einem vollständigen Wort, einem Wortbestandteil, einem Satzzeichen oder einer anderen Textsequenz entsprechen. Die Rune ist daher eine didaktische Darstellung eines Tokens und keine technische 1:1-Abbildung.

4. Vom Text zum Token

Bevor ein LLM Text verarbeiten kann, wird dieser durch einen Tokenizer in Tokens zerlegt.

Vereinfachtes Beispiel:

„Björn geht zum Orakel.“

→ [Björn] [geht] [zum] [Orakel] [.]

Diese Darstellung dient nur der Veranschaulichung. Ein realer Tokenizer kann denselben Text anders zerlegen.

Entscheidend ist: Token und Wort sind nicht dasselbe.

5. Ein Wort kann aus mehreren Tokens bestehen

Ein einzelnes menschliches Wort kann in mehrere Tokens aufgeteilt werden. Eine vereinfachte Darstellung wäre beispielsweise:

„Unverständlichkeit“ → [Un] [verständ] [lichkeit]

Die tatsächliche Aufteilung hängt vom verwendeten Tokenizer und dessen Vokabular ab.

Damit wird deutlich, warum die Aussage „Das Modell erzeugt Wort für Wort“ technisch zu grob ist. Die Verarbeitung und Erzeugung erfolgt auf Token-Ebene.

6. Was macht der Tokenizer?

Der Tokenizer bildet die Schnittstelle zwischen dem vom Menschen eingegebenen Text und der Verarbeitung durch das Modell.

Vereinfacht lässt sich die Verarbeitungskette so darstellen:

Text → Tokenizer → Tokenfolge → numerische Darstellung → LLM

Der Tokenizer entscheidet dabei, in welche Tokens die Textsequenz zerlegt wird. Die Tokens werden anschließend über ihre numerischen Repräsentationen für das Modell verarbeitbar.

7. Vom Token zur numerischen Darstellung

Tokens werden nicht als Zeichenketten wie „Björn“ oder „geht“ direkt in das neuronale Netz eingegeben. Sie werden zunächst über numerische Repräsentationen dargestellt. Dazu gehören insbesondere Embeddings.

Für dieses Modul genügt die grundlegende Vorstellung: Das Modell arbeitet intern mit Zahlen und Vektoren, nicht mit den menschlichen Wörtern selbst. Die mathematische Herleitung von Embeddings ist nicht Bestandteil dieses Moduls.

8. Was bedeutet Next Token Prediction?

Next Token Prediction beschreibt das grundlegende Generierungsprinzip eines autoregressiven Sprachmodells: Ausgehend vom bisherigen Kontext berechnet das Modell mögliche Fortsetzungen.

Beispiel:

„Björn geht zum ...“

Mögliche nächste Tokens könnten beispielsweise „Orakel“, „Hafen“ oder „Dorf“ sein. Das Modell weist diesen möglichen Fortsetzungen unterschiedliche Werte bzw. Wahrscheinlichkeiten zu.

Anschließend wird ein Token ausgewählt. Dieses Token wird an die bisherige Sequenz angehängt und bildet damit einen Teil des neuen Kontexts.

9. Der Kontext beeinflusst die Prediction

Die Vorhersage des nächsten Tokens hängt vom bisherigen Kontext ab.

Beispiele:

„Der Wikinger trägt einen ...“

→ mögliche Fortsetzung: „Helm“

„Der Wikinger segelt über einen ...“

→ mögliche Fortsetzung: „Fjord“

Die Wörter „Wikinger“ und „einen“ allein bestimmen die Ausgabe nicht. Entscheidend ist die gesamte für das Modell verfügbare Kontextinformation.

10. Die Antwort entsteht Token für Token

Eine generierte Antwort entsteht schrittweise. Vereinfacht:

„Du“ → „Du erreichst“ → „Du erreichst den“ → „Du erreichst den Gipfel“

Nach jedem ausgewählten Token verändert sich der Kontext. Das Modell berechnet deshalb für jeden weiteren Schritt erneut mögliche Fortsetzungen.

Die vollständige Antwort wird also nicht als fertiger Text in einem einzigen Schritt aus einer Datenbank abgerufen.

11. Prediction ist keine Wahrheitsprüfung

Die vom Modell berechnete Wahrscheinlichkeit beschreibt, wie gut ein Token aus Sicht des gelernten Modells zum bisherigen Kontext passt. Sie ist keine objektive Bewertung der Wahrheit einer Aussage.

Deshalb kann ein LLM auch eine sprachlich plausible, aber sachlich falsche Aussage erzeugen. Die Fähigkeit zur Vorhersage von Text ist nicht gleichbedeutend mit einer eingebauten Wahrheitsprüfung.

12. Logits und Wahrscheinlichkeiten

Das Modell erzeugt für die möglichen nächsten Tokens zunächst numerische Ausgabewerte, die als Logits bezeichnet werden. Aus diesen Werten kann eine Wahrscheinlichkeitsverteilung über die möglichen nächsten Tokens berechnet werden.

Vereinfacht:

Kontext → LLM → Logits → Wahrscheinlichkeitsverteilung → Decoding → nächstes Token

Für das Verständnis dieses Moduls ist keine mathematische Herleitung der Logits oder der verwendeten Funktionen erforderlich.

13. Wie wird das nächste Token ausgewählt?

Der Schritt, aus den möglichen Fortsetzungen das tatsächlich verwendete nächste Token auszuwählen, wird als Decoding bezeichnet.

Dabei gibt es unterschiedliche Verfahren. Grundsätzlich kann das jeweils wahrscheinlichste Token gewählt werden oder es kann aus einer Auswahl wahrscheinlicher Tokens gezogen werden.

- Greedy Decoding: bevorzugt das aktuell wahrscheinlichste Token.
- Sampling: wählt ein Token anhand der Wahrscheinlichkeitsverteilung aus.
- Temperature: beeinflusst, wie stark die Wahrscheinlichkeiten bei der Auswahl berücksichtigt werden.
- Top-k: beschränkt die Auswahl auf die k wahrscheinlichsten Tokens.
- Top-p / Nucleus Sampling: beschränkt die Auswahl auf eine Menge von Tokens mit einer gemeinsam betrachteten Wahrscheinlichkeitsmasse.

Die Verfahren werden hier nur konzeptionell betrachtet. Eine mathematische Untersuchung der Decoding-Verfahren ist nicht Bestandteil dieses Moduls.

14. Tokenisierung, Prediction und Decoding

Die drei Schritte erfüllen unterschiedliche Aufgaben:

- Tokenisierung: Wie wird der Eingabetext in Tokens zerlegt?
- Prediction: Welche möglichen Tokens können als Nächstes folgen?
- Decoding: Welches dieser möglichen Tokens wird ausgewählt?

Gesamtkette:

TEXT → TOKENISIERUNG → TOKENS → PREDICTION → MÖGLICHE NÄCHSTE TOKENS → DECODING → NÄCHSTES TOKEN → NEUER KONTEXT

Diese Kette wiederholt sich, bis die Generierung beendet wird.

15. Praxisbezug FISI

Das Token-Prinzip ist für die Systemintegration relevant, weil die Anzahl der Tokens Einfluss auf die technische Nutzung eines LLM haben kann.

- Context Window: Ein Modell kann nur eine begrenzte Menge an Kontext gleichzeitig verarbeiten.
- Speicher und Rechenleistung: Größere Kontextmengen und längere Ausgaben erhöhen den Verarbeitungsaufwand.
- Latenz: Mehr zu verarbeitende Tokens können die Antwortzeit beeinflussen.
- API-Nutzung und Kosten: Bei vielen LLM-Diensten wird die Nutzung unter anderem über Input- und Output-Tokens abgerechnet.
- RAG: Abgerufene Dokumentabschnitte werden ebenfalls in den Kontext des Modells eingebracht und erhöhen damit die Tokenmenge.
- Infrastruktur: Tokenmenge, Kontextlänge und Modellgröße wirken sich auf die Anforderungen an die Inferenzumgebung aus.

Vereinfacht lässt sich eine technische Verarbeitungskette darstellen als:

Benutzereingabe → Tokenizer → Tokens → Kontext → LLM/Inference → Logits → Decoding → Tokens → Tokenizer → Text

16. Typische Denkfehler

- „Ein Token ist immer ein Wort.“ – Falsch. Ein Wort kann aus mehreren Tokens bestehen; auch Satzzeichen oder Wortbestandteile können Tokens sein.
- „Das Modell sucht das nächste Wort in einem Wörterbuch.“ – Falsch. Das Modell berechnet mögliche nächste Tokens auf Grundlage des gelernten Modells und des aktuellen Kontexts.
- „Das wahrscheinlichste Token ist automatisch richtig.“ – Falsch. Wahrscheinlichkeit ist keine Wahrheitsprüfung.
- „Tokenisierung und Prediction sind dasselbe.“ – Falsch. Tokenisierung zerlegt Text; Prediction bewertet mögliche Fortsetzungen.
- „Die komplette Antwort wird auf einmal erzeugt.“ – Falsch. Die Generierung erfolgt schrittweise, Token für Token.
- „Bei derselben Eingabe muss immer exakt dieselbe Antwort entstehen.“ – Nicht zwingend. Decoding-Verfahren und Einstellungen können unterschiedliche Ausgaben ermöglichen.

17. Zusammenfassung

- Ein LLM verarbeitet Text auf der Ebene von Tokens.
- Ein Token kann einem Wort, einem Wortbestandteil, einem Satzzeichen oder einer anderen Textsequenz entsprechen.
- Der Tokenizer zerlegt den Eingabetext in eine Tokenfolge.
- Tokens werden für die Verarbeitung numerisch repräsentiert.
- Das Modell berechnet auf Grundlage des vorhandenen Kontexts mögliche nächste Tokens.
- Die Ausgabewerte des Modells können als Wahrscheinlichkeitsverteilung interpretiert werden.
- Ein Decoding-Verfahren bestimmt, welches Token als Nächstes verwendet wird.
- Das ausgewählte Token wird Teil des neuen Kontexts.

- Dieser Vorgang wiederholt sich, bis die Generierung beendet wird.

18. MERKE

Ein LLM erzeugt Text nicht Wort für Wort, sondern Token für Token.

Tokenisierung zerlegt Text in Tokens.

Prediction bewertet mögliche nächste Tokens.

Decoding bestimmt, welches Token verwendet wird.

Das ausgewählte Token wird Teil des neuen Kontexts.

19. Lerninhaltskontrolle

1. Was ist ein Token?
2. Warum sind Token und Wort nicht dasselbe?
3. Welche Aufgabe hat ein Tokenizer?
4. Kann ein einzelnes Wort aus mehreren Tokens bestehen? Begründe.
5. Was bedeutet Next Token Prediction?
6. Warum beeinflusst der Kontext die Vorhersage des nächsten Tokens?
7. Was sind Logits?
8. Was bezeichnet man beim LLM als Decoding?
9. Warum kann dieselbe Eingabe unterschiedliche Ausgaben erzeugen?
10. Was ist der Unterschied zwischen Tokenisierung und Prediction?

20. Lösungsschlüssel

1. Ein Token ist eine vom Modell verarbeitete Texteinheit. Ein Token kann beispielsweise ein ganzes Wort, ein Wortbestandteil oder ein Satzzeichen sein.
2. Weil ein Wort in mehrere Tokens zerlegt werden kann und ein Token nicht zwingend einem vollständigen Wort entspricht.
3. Der Tokenizer zerlegt Text in Tokens und bildet damit die Grundlage für die weitere Verarbeitung durch das Modell.
4. Ja. Die konkrete Aufteilung hängt vom verwendeten Tokenizer und dessen Vokabular ab.
5. Das Modell berechnet aus dem bisherigen Kontext mögliche Fortsetzungen und bewertet, welche Tokens als Nächstes folgen können.
6. Weil die bisherige Tokenfolge den Kontext bildet, auf dessen Grundlage das Modell die nächsten Tokens bewertet.

7. Logits sind die vom Modell erzeugten numerischen Ausgabewerte, aus denen eine Wahrscheinlichkeitsverteilung über mögliche nächste Tokens abgeleitet werden kann.
8. Decoding ist die Auswahl des nächsten Tokens aus den vom Modell bewerteten möglichen Fortsetzungen.
9. Weil unterschiedliche Decoding-Verfahren und Einstellungen wie Sampling oder Temperature unterschiedliche Auswahlentscheidungen ermöglichen können.
10. Tokenisierung zerlegt den Text in Tokens; Prediction bestimmt beziehungsweise bewertet mögliche nächste Tokens.